

Python Code

```
1  # =====
2  # Ethereum USD Historical Dataset - Statistical Analysis (Python Version)
3  # =====
4  # This script performs comprehensive statistical analysis on the Ethereum USD
5  # historical dataset including:
6  # - Descriptive Statistics (Mean, Mode, Median, SD, Variance, Quartiles, IQR, Range)
7  # - Data Visualization (Histograms, Box Plots, Density Plots, etc.)
8  # - Hypothesis Testing
9  # =====
10
11 import pandas as pd
12 import numpy as np
13 import matplotlib.pyplot as plt
14 import seaborn as sns
15 from scipy import stats
16 import os
17
18 # Set the style for plots
19 plt.style.use('seaborn-v0_8')
20 sns.set_palette("husl")
21
22 # Create output directory for plots
23 os.makedirs('plots', exist_ok=True)
24
25 # Load the dataset
26 print("Loading Ethereum USD Historical Dataset...")
27 data = pd.read_csv('../ethereum_usd_historical_dataset.csv')
28
29 # Display basic information about the dataset
30 print("\n=== DATASET OVERVIEW ===")
31 print(f"Number of rows: {len(data)}")
32 print(f"Number of columns: {len(data.columns)}")
33 print("\nColumn names:")
34 print(list(data.columns))
35
36 # Identify numeric columns
37 numeric_cols = data.select_dtypes(include=[np.number]).columns.tolist()
38 print(f"\n=== NUMERIC COLUMNS IDENTIFIED ===")
39 print(f"Total numeric columns: {len(numeric_cols)}")
40 print(numeric_cols)
41
42 # Remove non-meaningful numeric columns (like Year, Month, etc. that are identifiers)
43 # Keep actual data columns
44 meaningless_cols = ['Year', 'Month', 'Quarter', 'Week_Number', 'Day_of_Week']
45 meaningful_numeric_cols = [col for col in numeric_cols if col not in meaningless_cols]
46 print(f"\n=== MEANINGFUL NUMERIC COLUMNS FOR ANALYSIS ===")
47 print(meaningful_numeric_cols)
48
49 # =====
50 # FUNCTION DEFINITIONS
51 # =====
52
53 def get_mode(x):
54     """Calculate mode of a series, handling NaN values"""
55     x_clean = x.dropna()
56     if len(x_clean) == 0:
57         return np.nan
58     mode_result = x_clean.mode()
```

```

59     return mode_result.iloc[0] if not mode_result.empty else np.nan
60
61 def skewness_base(x):
62     """Calculate skewness using base Python (similar to R's moments package)"""
63     x_clean = x.dropna()
64     if len(x_clean) < 3:
65         return np.nan
66     m3 = np.sum((x_clean - np.mean(x_clean))**3) / len(x_clean)
67     m2 = np.sum((x_clean - np.mean(x_clean))**2) / len(x_clean)
68     return m3 / (m2**(3/2)) if m2 != 0 else 0
69
70 def kurtosis_base(x):
71     """Calculate kurtosis using base Python"""
72     x_clean = x.dropna()
73     if len(x_clean) < 4:
74         return np.nan
75     m4 = np.sum((x_clean - np.mean(x_clean))**4) / len(x_clean)
76     m2 = np.sum((x_clean - np.mean(x_clean))**2) / len(x_clean)
77     return m4 / (m2**2) - 3 if m2 != 0 else -3
78
79 def calculate_stats(series, col_name):
80     """Calculate comprehensive statistics for a column"""
81     x = series.dropna()
82     if len(x) == 0:
83         return pd.DataFrame({
84             'Column': [col_name],
85             'Mean': [np.nan],
86             'Median': [np.nan],
87             'Mode': [np.nan],
88             'Std_Dev': [np.nan],
89             'Variance': [np.nan],
90             'Q1': [np.nan],
91             'Q2': [np.nan],
92             'Q3': [np.nan],
93             'IQR': [np.nan],
94             'Min': [np.nan],
95             'Max': [np.nan],
96             'Range': [np.nan],
97             'Skewness': [np.nan],
98             'Kurtosis': [np.nan]
99         })
100
101     # Calculate statistics
102     mean_val = np.mean(x)
103     median_val = np.median(x)
104     mode_val = get_mode(x)
105     sd_val = np.std(x, ddof=1) # Sample standard deviation
106     var_val = np.var(x, ddof=1) # Sample variance
107     q1_val = np.percentile(x, 25)
108     q2_val = np.percentile(x, 50)
109     q3_val = np.percentile(x, 75)
110     iqr_val = q3_val - q1_val
111     min_val = np.min(x)
112     max_val = np.max(x)
113     range_val = max_val - min_val
114     skew_val = skewness_base(x)
115     kurt_val = kurtosis_base(x)
116
117     return pd.DataFrame({
118         'Column': [col_name],
119         'Mean': [mean_val],
120         'Median': [median_val],

```

```

121         'Mode': [mode_val],
122         'Std_Dev': [sd_val],
123         'Variance': [var_val],
124         'Q1': [q1_val],
125         'Q2': [q2_val],
126         'Q3': [q3_val],
127         'IQR': [iqr_val],
128         'Min': [min_val],
129         'Max': [max_val],
130         'Range': [range_val],
131         'Skewness': [skew_val],
132         'Kurtosis': [kurt_val]
133     })
134
135     # =====
136     # DESCRIPTIVE STATISTICS CALCULATION
137     # =====
138
139     print("\n\n")
140     print("#####")
141     print("#                DESCRIPTIVE STATISTICS ANALYSIS                #")
142     print("#####\n")
143
144     # Calculate statistics for all meaningful numeric columns
145     all_stats = pd.DataFrame()
146     for col in meaningful_numeric_cols:
147         if col in data.columns:
148             col_stats = calculate_stats(data[col], col)
149             all_stats = pd.concat([all_stats, col_stats], ignore_index=True)
150
151     # Display statistics
152     print("=== MEAN ===")
153     print(all_stats[['Column', 'Mean']].to_string(index=False))
154
155     print("\n=== MODE ===")
156     print(all_stats[['Column', 'Mode']].to_string(index=False))
157
158     print("\n=== MEDIAN ===")
159     print(all_stats[['Column', 'Median']].to_string(index=False))
160
161     print("\n=== STANDARD DEVIATION ===")
162     print(all_stats[['Column', 'Std_Dev']].to_string(index=False))
163
164     print("\n=== VARIANCE ===")
165     print(all_stats[['Column', 'Variance']].to_string(index=False))
166
167     print("\n=== QUARTILES ===")
168     print(all_stats[['Column', 'Q1', 'Q2', 'Q3']].to_string(index=False))
169
170     print("\n=== INTERQUARTILE RANGE (IQR) ===")
171     print(all_stats[['Column', 'IQR']].to_string(index=False))
172
173     print("\n=== RANGE ===")
174     print(all_stats[['Column', 'Min', 'Max', 'Range']].to_string(index=False))
175
176     # Save statistics to CSV
177     all_stats.to_csv('descriptive_statistics.csv', index=False)
178     print("\nDescriptive statistics saved to 'descriptive_statistics.csv'")
179
180     # Print full statistics table
181     print("\n=== COMPLETE STATISTICS TABLE ===")
182     print(all_stats.to_string(index=False))

```

```

183
184 # =====
185 # DATA VISUALIZATION
186 # =====
187
188 print("\n\n")
189 print("#####")
190 print("#                      DATA VISUALIZATION                      #")
191 print("#####\n")
192
193 # Select key fields for visualization
194 key_fields = ["Close", "High", "Low", "Open", "Volume", "Daily_Return_Pct",
195              "Log_Return", "Volatility_30d", "RSI_14", "MACD"]
196 key_fields = [field for field in key_fields if field in meaningful_numeric_cols]
197
198 # 1. HISTOGRAMS
199 print("Creating Histograms...")
200 for field in key_fields:
201     if field in data.columns:
202         plt.figure(figsize=(10, 8))
203         plt.hist(data[field].dropna(), bins=30, alpha=0.7, color='steelblue', edgecolor='white',
204 density=True)
205         plt.title(f'Histogram of {field}')
206         plt.xlabel(field)
207         plt.ylabel('Density')
208
209         # Add density curve
210         from scipy.stats import gaussian_kde
211         data_clean = data[field].dropna()
212         if len(data_clean) > 1:
213             kde = gaussian_kde(data_clean)
214             x_range = np.linspace(data_clean.min(), data_clean.max(), 1000)
215             plt.plot(x_range, kde(x_range), 'r-', linewidth=2, label='Density')
216
217             # Add mean and median lines
218             plt.axvline(np.mean(data_clean), color='green', linestyle='--', linewidth=2, label='Mean')
219             plt.axvline(np.median(data_clean), color='orange', linestyle='--', linewidth=2, label='Median')
220
221         plt.legend()
222         plt.tight_layout()
223         plt.savefig(f'plots/histogram_{field}.png', dpi=300, bbox_inches='tight')
224         plt.close()
225
226 # 2. BOX PLOTS
227 print("Creating Box Plots...")
228 for field in key_fields:
229     if field in data.columns:
230         plt.figure(figsize=(10, 8))
231         box_plot = plt.boxplot(data[field].dropna(), patch_artist=True)
232         box_plot['boxes'][0].set_facecolor('steelblue')
233         box_plot['boxes'][0].set_edgecolor('black')
234         plt.title(f'Box Plot of {field}')
235         plt.ylabel(field)
236
237         # Add mean point
238         mean_val = np.mean(data[field].dropna())
239         plt.scatter([1], [mean_val], color='red', s=100, zorder=3, label='Mean')
240         plt.legend()
241         plt.tight_layout()
242         plt.savefig(f'plots/boxplot_{field}.png', dpi=300, bbox_inches='tight')
243         plt.close()

```

```

244 # Combined box plot for multiple fields
245 plt.figure(figsize=(14, 10))
246 data_key = data[key_fields].dropna()
247 box_plot = plt.boxplot([data_key[col] for col in key_fields], patch_artist=True, tick_labels=key_fields)
248 colors = plt.cm.rainbow(np.linspace(0, 1, len(key_fields)))
249 for patch, color in zip(box_plot['boxes'], colors):
250     patch.set_facecolor(color)
251 plt.title('Box Plots of Key Ethereum Metrics')
252 plt.xticks(rotation=45)
253 plt.tight_layout()
254 plt.savefig('plots/boxplot_combined.png', dpi=300, bbox_inches='tight')
255 plt.close()
256
257 # 3. DENSITY PLOTS
258 print("Creating Density Plots...")
259 for field in key_fields:
260     if field in data.columns:
261         plt.figure(figsize=(10, 8))
262         data_clean = data[field].dropna()
263         if len(data_clean) > 1:
264             from scipy.stats import gaussian_kde
265             kde = gaussian_kde(data_clean)
266             x_range = np.linspace(data_clean.min(), data_clean.max(), 1000)
267             plt.plot(x_range, kde(x_range), 'b-', linewidth=2)
268             plt.fill_between(x_range, kde(x_range), alpha=0.3, color='steelblue')
269             plt.title(f'Density Plot of {field}')
270             plt.xlabel(field)
271             plt.ylabel('Density')
272         plt.tight_layout()
273         plt.savefig(f'plots/density_{field}.png', dpi=300, bbox_inches='tight')
274         plt.close()
275
276 # 4. BAR CHARTS (for categorical summaries)
277 print("Creating Bar Charts...")
278
279 # Monthly average close price
280 if 'Month' in data.columns and 'Close' in data.columns:
281     monthly_avg = data.groupby('Month')['Close'].mean().reset_index()
282     plt.figure(figsize=(10, 8))
283     plt.bar(monthly_avg['Month'], monthly_avg['Close'], color='steelblue', edgecolor='white')
284     plt.title('Average Close Price by Month')
285     plt.xlabel('Month')
286     plt.ylabel('Average Close Price')
287     plt.tight_layout()
288     plt.savefig('plots/bar_monthly_avg_close.png', dpi=300, bbox_inches='tight')
289     plt.close()
290
291 # Yearly average close price
292 if 'Year' in data.columns and 'Close' in data.columns:
293     yearly_avg = data.groupby('Year')['Close'].mean().reset_index()
294     plt.figure(figsize=(10, 8))
295     plt.bar(yearly_avg['Year'], yearly_avg['Close'], color='coral', edgecolor='white')
296     plt.title('Average Close Price by Year')
297     plt.xlabel('Year')
298     plt.ylabel('Average Close Price')
299     plt.tight_layout()
300     plt.savefig('plots/bar_yearly_avg_close.png', dpi=300, bbox_inches='tight')
301     plt.close()
302
303 # 5. SCATTER PLOTS
304 print("Creating Scatter Plots...")
305

```

```

306 # Close vs Volume
307 if 'Close' in data.columns and 'Volume' in data.columns:
308     plt.figure(figsize=(10, 8))
309     plt.scatter(data['Close'], data['Volume'], alpha=0.5, color='steelblue', s=20)
310     plt.title('Close Price vs Volume')
311     plt.xlabel('Close Price')
312     plt.ylabel('Volume')
313
314     # Add trend line
315     z = np.polyfit(data['Close'].dropna(), data['Volume'].dropna(), 1)
316     p = np.poly1d(z)
317     plt.plot(data['Close'], p(data['Close']), "r--", alpha=0.8, linewidth=2)
318     plt.tight_layout()
319     plt.savefig('plots/scatter_close_volume.png', dpi=300, bbox_inches='tight')
320     plt.close()
321
322 # Close vs Daily Return
323 if 'Close' in data.columns and 'Daily_Return_Pct' in data.columns:
324     valid_idx = data['Close'].notna() & data['Daily_Return_Pct'].notna()
325     plt.figure(figsize=(10, 8))
326     plt.scatter(data.loc[valid_idx, 'Close'], data.loc[valid_idx, 'Daily_Return_Pct'],
327                alpha=0.7, color='coral', s=20)
328     plt.title('Close Price vs Daily Return')
329     plt.xlabel('Close Price')
330     plt.ylabel('Daily Return (%)')
331
332     # Add trend line
333     z = np.polyfit(data.loc[valid_idx, 'Close'], data.loc[valid_idx, 'Daily_Return_Pct'], 1)
334     p = np.poly1d(z)
335     plt.plot(data.loc[valid_idx, 'Close'], p(data.loc[valid_idx, 'Close']), "b--", alpha=0.8, linewidth=2)
336     plt.tight_layout()
337     plt.savefig('plots/scatter_close_daily_return.png', dpi=300, bbox_inches='tight')
338     plt.close()
339
340 # 6. CORRELATION MATRIX HEATMAP
341 print("Creating Correlation Heatmap...")
342 plt.figure(figsize=(12, 10))
343 cor_matrix = data[key_fields].corr()
344 sns.heatmap(cor_matrix, annot=True, cmap='coolwarm', center=0, square=True, linewidths=0.5)
345 plt.title('Correlation Matrix of Key Ethereum Metrics')
346 plt.tight_layout()
347 plt.savefig('plots/correlation_heatmap.png', dpi=300, bbox_inches='tight')
348 plt.close()
349
350 # Save correlation matrix
351 cor_matrix.to_csv('correlation_matrix.csv')
352 print("Correlation matrix saved to 'correlation_matrix.csv'")
353
354 # 7. Q-Q PLOTS for key fields
355 print("Creating Q-Q Plots...")
356 for field in ["Close", "Daily_Return_Pct", "Volume"]:
357     if field in data.columns:
358         plt.figure(figsize=(10, 8))
359         data_clean = data[field].dropna()
360         if len(data_clean) > 0:
361             from scipy import stats as sp_stats
362             sp_stats.probplot(data_clean, dist="norm", plot=plt)
363             plt.title(f'Q-Q Plot of {field}')
364             plt.tight_layout()
365             plt.savefig(f'plots/qqplot_{field}.png', dpi=300, bbox_inches='tight')
366             plt.close()
367

```

```

368 print("\nAll visualizations saved to 'plots/' directory")
369
370 # =====
371 # HYPOTHESIS TESTING
372 # =====
373
374 print("\n\n")
375 print("#####")
376 print("#                      HYPOTHESIS TESTING                      #")
377 print("#####\n")
378
379 # Test 1: One-sample t-test - Is the mean daily return significantly different from 0?
380 print("\n=== HYPOTHESIS TEST 1: One-Sample T-Test ===")
381 print("Question: Is the mean daily return significantly different from 0?\n")
382 print("H0: Mean daily return = 0")
383 print("H1: Mean daily return != 0\n")
384
385 daily_return = data['Daily_Return_Pct'].dropna()
386 t_test_result = stats.ttest_1samp(daily_return, 0)
387 print("Test Results:")
388 print(f"t-statistic: {t_test_result.statistic:.6f}")
389 print(f"p-value: {t_test_result.pvalue:.6f}")
390 print(f"degrees of freedom: {len(daily_return)-1}")
391
392 if t_test_result.pvalue < 0.05:
393     print("\nConclusion: At 5% significance level, we REJECT the null hypothesis.")
394     print("The mean daily return is significantly different from 0.")
395 else:
396     print("\nConclusion: At 5% significance level, we FAIL TO REJECT the null hypothesis.")
397     print("There is no significant evidence that the mean daily return is different from 0.")
398
399 # Test 2: Two-sample t-test - Is there a significant difference between Close prices in different years?
400 print("\n\n=== HYPOTHESIS TEST 2: Two-Sample T-Test ===")
401 print("Question: Is there a significant difference in Close prices between 2021 and 2022?\n")
402
403 close_2021 = data[data['Year'] == 2021]['Close'].dropna()
404 close_2022 = data[data['Year'] == 2022]['Close'].dropna()
405
406 print("H0: Mean Close price in 2021 = Mean Close price in 2022")
407 print("H1: Mean Close price in 2021 != Mean Close price in 2022\n")
408 print("Sample sizes:")
409 print(f"2021: {len(close_2021)}")
410 print(f"2022: {len(close_2022)}\n")
411
412 t_test_2 = stats.ttest_ind(close_2021, close_2022, equal_var=False)
413 print("Test Results:")
414 print(f"t-statistic: {t_test_2.statistic:.6f}")
415 print(f"p-value: {t_test_2.pvalue:.6f}")
416 print(f"degrees of freedom: {t_test_2.df:.2f}")
417
418 if t_test_2.pvalue < 0.05:
419     print("\nConclusion: At 5% significance level, we REJECT the null hypothesis.")
420     print("There is a significant difference in Close prices between 2021 and 2022.")
421 else:
422     print("\nConclusion: At 5% significance level, we FAIL TO REJECT the null hypothesis.")
423     print("There is no significant difference in Close prices between 2021 and 2022.")
424
425 # Test 3: Correlation Test - Is there a significant correlation between Close price and Volume?
426 print("\n\n=== HYPOTHESIS TEST 3: Correlation Test ===")
427 print("Question: Is there a significant correlation between Close price and Volume?\n")
428
429 valid_corr = data[['Close', 'Volume']].dropna()

```



```

430 close_corr = valid_corr['Close']
431 volume_corr = valid_corr['Volume']
432
433 print("H0: Correlation between Close and Volume = 0")
434 print("H1: Correlation between Close and Volume != 0\n")
435
436 cor_test_result = stats.pearsonr(close_corr, volume_corr)
437 print("Test Results:")
438 print(f"Correlation coefficient: {cor_test_result[0]:.6f}")
439 print(f"p-value: {cor_test_result[1]:.6f}")
440
441 if cor_test_result[1] < 0.05:
442     print("\nConclusion: At 5% significance level, we REJECT the null hypothesis.")
443     print("There is a significant correlation between Close price and Volume.")
444 else:
445     print("\nConclusion: At 5% significance level, we FAIL TO REJECT the null hypothesis.")
446     print("There is no significant correlation between Close price and Volume.")
447
448 # Test 4: Paired t-test - Is there a significant difference between High and Low prices?
449 print("\n\n=== HYPOTHESIS TEST 4: Paired T-Test ===")
450 print("Question: Is there a significant difference between High and Low prices?\n")
451
452 print("H0: Mean High price = Mean Low price")
453 print("H1: Mean High price != Mean Low price\n")
454
455 paired_test = stats.ttest_rel(data['High'].dropna(), data['Low'].dropna())
456 print("Test Results:")
457 print(f"t-statistic: {paired_test.statistic:.6f}")
458 print(f"p-value: {paired_test.pvalue:.6f}")
459 print(f"degrees of freedom: {len(data)-1}")
460
461 if paired_test.pvalue < 0.05:
462     print("\nConclusion: At 5% significance level, we REJECT the null hypothesis.")
463     print("There is a significant difference between High and Low prices.")
464 else:
465     print("\nConclusion: At 5% significance level, we FAIL TO REJECT the null hypothesis.")
466     print("There is no significant difference between High and Low prices.")
467
468 # Test 5: Chi-Square Test - Is there a relationship between Day of Week and Price Movement?
469 print("\n\n=== HYPOTHESIS TEST 5: Chi-Square Test ===")
470 print("Question: Is there a significant relationship between Day of Week and Price Movement direction?\n")
471
472 # Create price movement categories (only for non-NA values)
473 valid_idx = data['Daily_Return_Pct'].notna()
474 price_movement = np.where(data.loc[valid_idx, 'Daily_Return_Pct'] > 0, "Up",
475                             np.where(data.loc[valid_idx, 'Daily_Return_Pct'] < 0, "Down", "Neutral"))
476 day_of_week = data.loc[valid_idx, 'Day_of_Week']
477
478 # Create contingency table
479 contingency_table = pd.crosstab(day_of_week, price_movement)
480 print("Contingency Table:")
481 print(contingency_table)
482
483 print("\nH0: Day of Week and Price Movement are independent")
484 print("H1: Day of Week and Price Movement are not independent\n")
485
486 chi_square_result = stats.chi2_contingency(contingency_table)
487 print("Test Results:")
488 print(f"Chi-square statistic: {chi_square_result[0]:.6f}")
489 print(f"p-value: {chi_square_result[1]:.6f}")
490 print(f"degrees of freedom: {chi_square_result[2]}")
491

```



```

492 if chi_square_result[1] < 0.05:
493     print("\nConclusion: At 5% significance level, we REJECT the null hypothesis.")
494     print("There is a significant relationship between Day of Week and Price Movement.")
495 else:
496     print("\nConclusion: At 5% significance level, we FAIL TO REJECT the null hypothesis.")
497     print("Day of Week and Price Movement appear to be independent.")
498
499 # Save hypothesis test results
500 with open('hypothesis_test_results.txt', 'w', encoding='utf-8') as f:
501     f.write("HYPOTHESIS TEST RESULTS\n")
502     f.write("=====\n\n")
503
504     f.write("Test 1: One-Sample T-Test (Daily Return)\n")
505     f.write("-----\n")
506     f.write(f"t-statistic: {t_test_result.statistic:.6f}\n")
507     f.write(f"p-value: {t_test_result.pvalue:.6f}\n")
508     f.write(f"df: {len(daily_return)-1}\n")
509     f.write("\n")
510
511     f.write("Test 2: Two-Sample T-Test (2021 vs 2022 Close Prices)\n")
512     f.write("-----\n")
513     f.write(f"t-statistic: {t_test_2.statistic:.6f}\n")
514     f.write(f"p-value: {t_test_2.pvalue:.6f}\n")
515     f.write(f"df: {t_test_2.df:.2f}\n")
516     f.write("\n")
517
518     f.write("Test 3: Correlation Test (Close vs Volume)\n")
519     f.write("-----\n")
520     f.write(f"Correlation coefficient: {cor_test_result[0]:.6f}\n")
521     f.write(f"p-value: {cor_test_result[1]:.6f}\n")
522     f.write("\n")
523
524     f.write("Test 4: Paired T-Test (High vs Low)\n")
525     f.write("-----\n")
526     f.write(f"t-statistic: {paired_test.statistic:.6f}\n")
527     f.write(f"p-value: {paired_test.pvalue:.6f}\n")
528     f.write(f"df: {len(data)-1}\n")
529     f.write("\n")
530
531     f.write("Test 5: Chi-Square Test (Day of Week vs Price Movement)\n")
532     f.write("-----\n")
533     f.write(f"Chi-square statistic: {chi_square_result[0]:.6f}\n")
534     f.write(f"p-value: {chi_square_result[1]:.6f}\n")
535     f.write(f"df: {chi_square_result[2]}\n")
536
537 print("\nHypothesis test results saved to 'hypothesis_test_results.txt'")
538 print("\nAnalysis complete!")
539

```